CPU Scheduling Simulator

Team Members

Name	ID
Ahmed Elsayed Allam	223102168
Ahmed Khaled Hassona	223104027
Ahmed Fathy Essa	223106563
Mariam Mohamed Othman	223105468
Salma Mohamed Nasef	223103501

CPU Scheduling Simulator

---

1. Project Overview

Purpose

This project implements a **CPU Scheduling Simulator** that emulates how an operating system schedules processes using different algorithms. It demonstrates process management, inter-process communication (IPC), and scheduling strategies.

Key Features

- **Three Scheduling Algorithms**: HPF (Highest Priority First), SJN (Shortest Job Next), RR (Round Robin)
- **Real-time Simulation**: Uses simulated clock with shared memory
- **Process Management**: Fork/exec model with process lifecycle tracking
- **IPC Mechanisms**: Message queues, shared memory, signals
- **Performance Analysis**: Calculates CPU utilization, waiting time, turnaround time, WTA
- **Interactive GUI**: HTML/JavaScript visualization tool

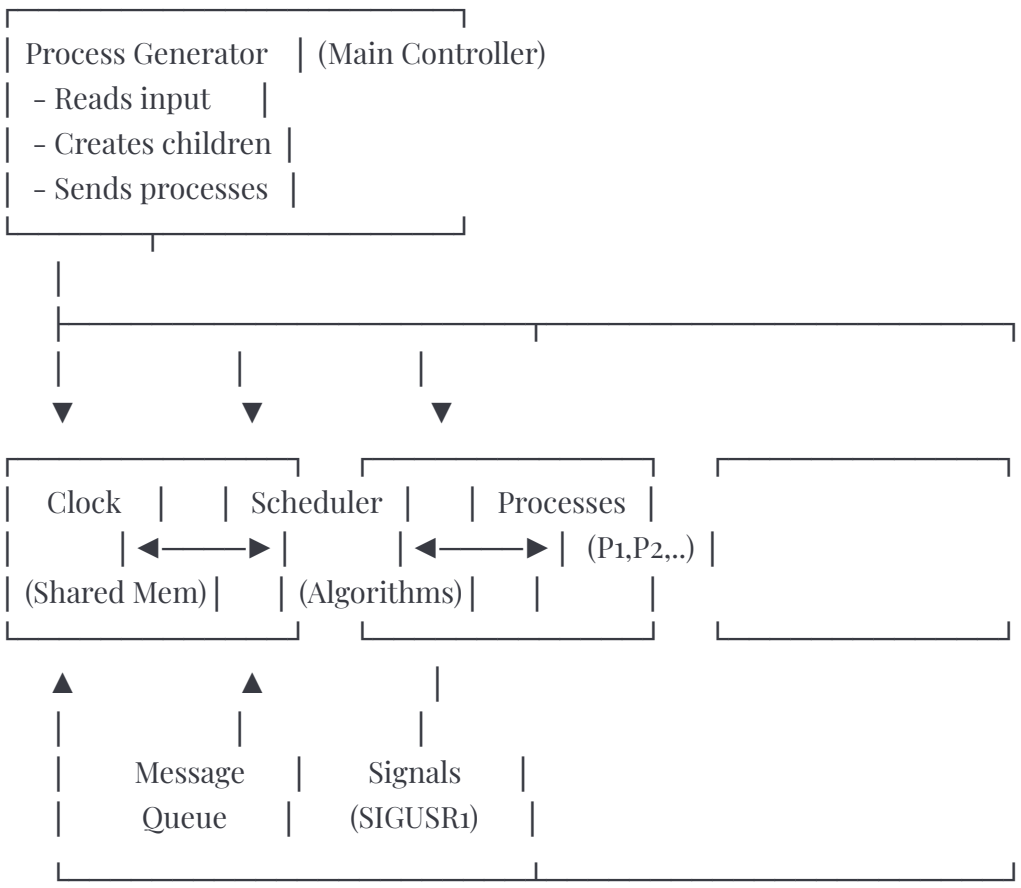
Technologies Used

- **Language**: C (POSIX-compliant)
- **IPC**: System V IPC (message queues, shared memory)
- **Process Control**: fork(), exec(), signals
- **GUI**: HTML5, CSS3, JavaScript (ES6)

---

2. System Architecture

Component Diagram



3. File Structure

Source Files Overview

File	Lines	Purpose
clk.c	50	Emulated system clock using shared memory
process_generator.c	250	Main controller, creates all processes
scheduler.c	450	Core scheduler with queue management
process.c	60	Simulates CPU-bound process execution
hpf_scheduler.c	25	Highest Priority First algorithm
sjn_scheduler.c	25	Shortest Job Next algorithm
rr_scheduler.c	30	Round Robin algorithm
test_generator.c	40	Generates random test cases
GUI.html	1400	Interactive web visualization

4. Header Files (Inferred)

headers.h

c

```c
#ifndef HEADERS_H
#define HEADERS_H

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/msg.h>
#include <sys/wait.h>
#include <stdbool.h>

#define SHKEY 300  // Shared memory key for clock

// Clock functions
void initClk();         // Attach to shared memory clock
int getClk();           // Get current simulation time
void destroyClk(bool);  // Detach (and optionally destroy) clock


#endif
```

CPU Scheduling Simulator

---

schedule.h

c

```c
#ifndef SCHEDULE_H
#define SCHEDULE_H

typedef enum { READY, RUNNING, FINISHED } ProcessState;

typedef struct {
    int id, arrivalTime, runtime, priority;
    int remainingTime, waitingTime, executionTime;
    int startTime, finishTime, lastStopTime;
    pid_t pid;
    ProcessState state;
    bool started;
} PCB;

typedef struct QueueNode {
    PCB* pcb;
    struct QueueNode* next;
} QueueNode;

typedef struct {
    QueueNode *head, *tail;
    int size;
} Queue;

// Queue operations
void initQueue(Queue* q);
void enqueue(Queue* q, PCB* pcb);
PCB* dequeue(Queue* q);
PCB* peek(Queue* q);
bool isEmpty(Queue* q);
void removeFromQueue(Queue* q, PCB* pcb);
```

```
// Scheduler functions
void receiveProcesses();
void selectNextProcess();
void startProcess(PCB* pcb);
void stopProcess(PCB* pcb);
void resumeProcess(PCB* pcb);
void finishProcess(PCB* pcb);
void handleProcessFinish(int signum);
void writeLog(const char* state, PCB* pcb);
void writePerformanceMetrics();
void cleanup();

// Algorithm selectors
PCB* selectHPF();
PCB* selectSJN();
PCB* selectRR();

#endif
```

5. Core Components

5.1 Clock Process (clk.c)

Purpose: Centralized time source using shared memory.

Main Function:

c

```c
int main(int argc, char *argv[]) {
    signal(SIGINT, cleanup);
    shmid = shmget(SHKEY, 4, IPC_CREAT | 0644);
    int *shmaddr = (int *)shmat(shmid, (void *)0, 0);
    *shmaddr = 0;

    while (1) {
        sleep(1);      // 1 real second
        (*shmaddr)++;  // Increment time
    }
}
```

Key Points:

- Creates 4-byte shared memory segment
- Increments every second
- All processes read same value
- Signal handler cleans up on SIGINT

5.2 Process Generator (process_generator.c)

Main Workflow:

1. Read processes.txt
2. Get algorithm choice from user
3. Create message queue
4. Fork clock process
5. Fork scheduler process
6. Send processes at arrival times
7. Send termination signal
8. Wait and cleanup

Key Functions:

readProcesses(): Parses input file

```
c
while (fgets(line, sizeof(line), file)) {
    if (line[0] == '#' || line[0] == '\n') continue;
    sscanf(line, "%d\t%d\t%d\t%d", &id, &arrival, &runtime, &priority);
    processes[count++] = {id, arrival, runtime, priority};
}
```

sendProcessesToScheduler(): Timing-based delivery

```
c
while (currentProcess < count) {
    currentTime = getClk();
    while (currentProcess < count &&
           processes[currentProcess].arrivalTime <= currentTime) {
        msg.mtype = 1;
        msg.process = processes[currentProcess];
        msgsnd(msgqid, &msg, sizeof(Process), 0);
        currentProcess++;
    }
```

```
    usleep(100000);  // Avoid busy-wait
}
```

5.3 Scheduler (scheduler.c)

Main Loop:

```
c
while (!allDone || !isEmpty(&readyQueue) || runningProcess) {
    currentTime = getClk();
    receiveProcesses();         // Get new arrivals

    if (runningProcess && runningProcess->remainingTime <= 0)
        finishProcess(runningProcess);

    if (algorithm == 3 && quantumCounter >= quantum)
        preemptForQuantum();

    if (!runningProcess && !isEmpty(&readyQueue))
        selectNextProcess();

    checkTerminationSignal();
    usleep(10000);
}
```

Process Management:

startProcess(): Fork and exec

```
c
pid = fork();
if (pid == 0) {
    sprintf(arg, "%d", pcb->remainingTime);
```

```
    execl("./process.out", "process.out", arg, NULL);
}
pcb->pid = pid;
pcb->state = RUNNING;
pcb->startTime = getClk();
```

stopProcess(): Preempt with SIGSTOP

```c
kill(pcb->pid, SIGSTOP);
pcb->state = READY;
pcb->lastStopTime = getClk();
```

resumeProcess(): Continue with SIGCONT

```c
kill(pcb->pid, SIGCONT);
pcb->state = RUNNING;
pcb->waitingTime += (getClk() - pcb->lastStopTime);
```

finishProcess(): Calculate metrics and terminate

```c
pcb->finishTime = getClk();
pcb->turnaroundTime = pcb->finishTime - pcb->arrivalTime;
pcb->wta = (double)pcb->turnaroundTime / pcb->runtime;
kill(pcb->pid, SIGKILL);
waitpid(pcb->pid, NULL, 0);
```

5.4 Process Simulation (process.c)

Purpose: Simulates CPU-bound execution.

CPU Scheduling Simulator

---

c

```c
int main(int argc, char *argv[]) {
    initClk();
    remainingtime = atoi(argv[1]);
    int lastTime = getClk();

    while (remainingtime > 0) {
        int currentTime = getClk();
        if (currentTime > lastTime) {
            remainingtime -= (currentTime - lastTime);
            lastTime = currentTime;
        }
    }

    kill(getppid(), SIGUSR1);  // Notify completion
    destroyClk(false);
    return 0;
}
```

---

6. Scheduling Algorithms

6.1 HPF (Highest Priority First) - Preemptive

File: hpf_scheduler.c

Logic: Select process with lowest priority number.

c
```
PCB* selectHPF() {
    PCB* highest = NULL;
    for (node = readyQueue.head; node; node = node->next) {
        if (!highest || node->pcb->priority < highest->priority ||
            (node->pcb->priority == highest->priority &&
             node->pcb->arrivalTime < highest->arrivalTime)) {
            highest = node->pcb;
        }
    }
    return highest;
}
```

Characteristics:

- **Preemptive**: Higher priority interrupts
- **Tie-break**: Earlier arrival
- **Use case**: Real-time systems
- **Issue**: Starvation of low priority

6.2 SJN (Shortest Job Next) - Non-Preemptive

File: sjn_scheduler.c

Logic: Select process with shortest remaining time.

```c
PCB* selectSJN() {
    PCB* shortest = NULL;
    for (node = readyQueue.head; node; node = node->next) {
        if (!shortest || node->pcb->remainingTime < shortest->remainingTime ||
            (node->pcb->remainingTime == shortest->remainingTime &&
             node->pcb->arrivalTime < shortest->arrivalTime)) {
            shortest = node->pcb;
        }
    }
    return shortest;
}
```

Characteristics:

- **Non-preemptive**: Runs to completion
- **Optimal**: Minimizes avg waiting time
- **Issue**: Long process starvation

6.3 RR (Round Robin)

File: rr_scheduler.c

Logic: FIFO with time quantum.

```c
PCB* selectRR() {
    PCB* selected = peek(&readyQueue);
    // Rotate queue
    if (readyQueue.size > 1) {
        QueueNode* head = readyQueue.head;
        readyQueue.head = head->next;
        readyQueue.tail->next = head;
        head->next = NULL;
```

```
        readyQueue.tail = head;
    }
    return selected;
}
```

Quantum Handling (in scheduler.c):

c

```
if (algorithm == 3 && runningProcess) {
    quantumCounter++;
    if (quantumCounter >= quantum && runningProcess->remainingTime > 0) {
        stopProcess(runningProcess);
        enqueue(&readyQueue, runningProcess);
        runningProcess = NULL;
        quantumCounter = 0;
    }
}
```

Characteristics:

- **Fair**: All get equal time slices
- **Preemptive**: After quantum expires
- **Good**: Response time
- **Trade-off**: Context switch overhead

---

7. Inter-Process Communication

7.1 Shared Memory (Clock)

Creation:

c
```
shmid = shmget(SHKEY, 4, IPC_CREAT | 0644);
int *shmaddr = (int *)shmat(shmid, (void *)0, 0);
```

Access:

c
```
void initClk() {
    shmid = shmget(SHKEY, 4, 0444);
    shmaddr = (int *)shmat(shmid, (void *)0, 0);
}

int getClk() { return *shmaddr; }
```

Cleanup:

c
```
shmdt(shmaddr);            // Detach
shmctl(shmid, IPC_RMID, NULL); // Remove
```

7.2 Message Queues

Creation:

c
```
key_t msgkey = ftok(".", 'M');
msgqid = msgget(msgkey, IPC_CREAT | 0644);
```

Message Structure:

```c
typedef struct {
    long mtype;      // 1=process, 2=terminate
    Process process;
} Message;
```

Send:

```c
msg.mtype = 1;
msg.process = currentProcess;
msgsnd(msgqid, &msg, sizeof(Process), 0);
```

Receive (non-blocking):

```c
while (msgrcv(msgqid, &msg, sizeof(Process), 1, IPC_NOWAIT) != -1) {
    // Process message
}
```

7.3 Signals

Signal	From	To	Purpose
SIGINT	User	All	Graceful shutdown
SIGUSR1	Process	Scheduler	Completion notification
SIGSTOP	Scheduler	Process	Preempt (pause)

CPU Scheduling Simulator

SIGCONT	Scheduler	Process	Resume
SIGKILL	Scheduler	Process	Terminate

8. Build and Execution

Inferred Makefile

makefile

```
CC = gcc
CFLAGS = -Wall -g
INCLUDES = -I./include

all: clk.out process_generator.out scheduler.out process.out test_generator.out

clk.out: src/clk.c
	$(CC) $(CFLAGS) $(INCLUDES) src/clk.c -o clk.out

process_generator.out: src/process_generator.c
	$(CC) $(CFLAGS) $(INCLUDES) src/process_generator.c -o process_generator.out

scheduler.out: src/scheduler.c algorithms/src/hpf_scheduler.c \
	 algorithms/src/sjn_scheduler.c algorithms/src/rr_scheduler.c
	$(CC) $(CFLAGS) $(INCLUDES) src/scheduler.c \
		algorithms/src/hpf_scheduler.c \
		algorithms/src/sjn_scheduler.c \
		algorithms/src/rr_scheduler.c \
		-lm -o scheduler.out

process.out: src/process.c
	$(CC) $(CFLAGS) $(INCLUDES) src/process.c -o process.out

test_generator.out: src/test_generator.c
	$(CC) $(CFLAGS) src/test_generator.c -o test_generator.out

clean:
	rm -f *.out scheduler.log scheduler.perf

run: all
```

CPU Scheduling Simulator

./process_generator.out

Compilation Steps

bash
 # Compile all
make

 # Or individually
gcc -Wall -I./include src/clk.c -o clk.out
gcc -Wall -I./include src/process_generator.c -o process_generator.out
gcc -Wall -I./include src/scheduler.c algorithms/src/*.c -lm -o scheduler.out

gcc -Wall -I./include src/process.c -o process.out

Execution

bash
 # Run simulation
./process_generator.out

 # Output:
 # 1. Choose algorithm (1-3)
 # 2. Enter quantum (if RR)
 # 3. Simulation runs

 # 4. Creates scheduler.log and scheduler.perf

---

9. Input/Output Format

Input File (processes.txt)

Format:

```
#id arrival runtime priority
1   2       1       10
2   12      23      2
3   16      29      0
4   16      9       5
5   19      13      9
6   21      28      2
```

Fields:

- **id**: Unique process identifier
- **arrival**: Arrival time in simulation
- **runtime**: CPU burst time needed
- **priority**: Priority level (lower = higher priority)

Output File (scheduler.log)

Format:

```
#At time x process y state arr w total z remain y wait k
At time 2 process 1 started arr 2 total 1 remain 1 wait 0
At time 3 process 1 finished arr 2 total 1 remain 0 wait 0 TA 1 WTA 1.00
At time 12 process 2 started arr 12 total 23 remain 23 wait 0
At time 16 process 3 started arr 16 total 29 remain 29 wait 0
At time 16 process 2 stopped arr 12 total 23 remain 19 wait 0
...
```

CPU Scheduling Simulator

States:

- **started**: Process begins first execution
- **stopped**: Process preempted
- **resumed**: Process continues after stop
- **finished**: Process completed (includes TA and WTA)

Performance File (scheduler.perf)

Format:

CPU utilization = 95.83%
Avg WTA = 1.67
Avg Waiting = 5.33

Std WTA = 0.85

Metrics:

- **CPU utilization**: (Total burst time / Total time) × 100
- **Avg WTA**: Average Weighted Turnaround Time
- **Avg Waiting**: Average time in ready queue
- **Std WTA**: Standard deviation of WTA

---

10. Performance Metrics

Formulas

Turnaround Time (TA):

TA = Finish Time - Arrival Time

Waiting Time:

Waiting Time = TA - Burst Time

Weighted Turnaround Time (WTA):

WTA = TA / Burst Time

CPU Utilization:

CPU Util = (Sum of Burst Times / Total Simulation Time) × 100%

Throughput:

Throughput = Number of Processes / (Max Finish - Min Arrival)

Standard Deviation of WTA:

Variance = $(\Sigma \text{WTA}^2) / n - (\text{Avg WTA})^2$

Std Dev = $\sqrt{\text{Variance}}$

Implementation (in scheduler.c)

c

```c
void writePerformanceMetrics() {
    int n = finishedCount;
    double avgWTA = totalWTA / n;
    double avgWaiting = (double)totalWaitingTime / n;
```

```
    double variance = (totalWTASquared / n) - (avgWTA * avgWTA);
    double stdWTA = sqrt(variance);
    double cpuUtil = ((double)totalRuntime / currentTime) * 100;

    fprintf(perfFile, "CPU utilization = %.2f%%\n", cpuUtil);
    fprintf(perfFile, "Avg WTA = %.2f\n", avgWTA);
    fprintf(perfFile, "Avg Waiting = %.2f\n", avgWaiting);
    fprintf(perfFile, "Std WTA = %.2f\n", stdWTA);
}
```

CPU Scheduling Simulator

Tasks

Task 0: Additional Scheduling Algorithm

Objective: Implement a new scheduling algorithm to expand beyond existing RR, HPFS, and SJN schedulers.

Algorithm Options:

- Preemptive Shortest Job First (SJF)
- Multilevel Feedback Queue (MLFQ)
- First-Come, First-Served (FCFS)
- Lottery Scheduling

Requirements:

- Research and document algorithm pros/cons and use cases
- Implement with modular design for easy comparison
- Generate .perf files with metrics: CPU utilization, Avg WTA, Avg Waiting, Std WTA
- Test on same cases as existing algorithms
- Validate calculations manually for small test cases

Task 1: GUI Development with Sciter

Objective: Build a GUI/WebUI using Sciter (HTML/CSS/JS) integrated with C codebase.

Challenges:

- Team has no HTML/CSS/Sciter experience
- Limited C documentation (most examples are C++)
- Fallback plan: Qt with C++ if Sciter fails

Goals:

- Create minimal viable UI using only HTML/CSS (avoid JavaScript)
- Build proof-of-concept (basic window with buttons/text fields)
- Document the learning process

Steps:

1. Download Sciter SDK and integrate with C project
2. Adapt C++ tutorials for C
3. Design simple HTML/CSS template
4. Test UI responsiveness and debug integration issues

Task 2: Unit Testing with Google Test

Objective: Implement comprehensive unit testing for functional correctness.

Framework: Google Test (gtest) - C++ framework compatible with C via wrappers

Steps:

1. Install and configure gtest with project
2. Write unit tests focusing on:
    - Core logic and critical functions
    - Edge cases and error handling
    - Input validation and boundary conditions
3. Integrate with CI/CD pipeline (optional)
4. Document test cases and expected outcomes

Coverage Goal: Minimum 80% for critical paths (measure with gcov/lcov)

Task 2.1: Memory Leak Detection

Objective: Identify and fix memory leaks and security vulnerabilities.

Tools:

- **Valgrind** (Linux/MacOS): Full leak detection with invalid access tracking
- **AddressSanitizer (ASan)**: Fast compiler-based detection
- **Static Analysis**: clang-tidy or cppcheck

Process:

1. Run leak detection tools on entire codebase
2. Document all leaks in MEMORY_ISSUES.md
3. Fix each leak by ensuring malloc/free pairs on all code paths
4. Implement defensive programming (set pointers to NULL after free)
5. Add CI/CD automation for continuous testing

Common Issues to Watch:

- Mismatched allocation/deallocation
- Leaks in error paths
- Global variable cleanup

Task 2.2: Replace Unsafe C Functions

Objective: Replace unsafe functions with secure alternatives and optimize code.

Key Replacements:

- scanf ↠ fgets + sscanf
- printf ↠ snprintf
- gets ↠ fgets
- strcpy ↠ strncpy
- strcat ↠ strncat
- sprintf ↠ snprintf

Security Focus:

- Validate all inputs and check return values
- Use bounds-checked functions
- Apply static analysis tools

Performance Optimizations:

- Use compiler flags: `-O2` or `-O3`
- Pass pointers instead of large structs
- Use `restrict` keyword for pointer aliases
- Consider inline functions for frequent calls
- Implement cache-aware coding patterns

Deliverable: Create SAFE_CODING.md with team guidelines

Task 3: FlameGraphs for Performance Profiling

Objective: Visualize performance bottlenecks using FlameGraphs (optional but valuable).

Tools:

- **perf** (Linux): Collect stack traces
- **FlameGraph scripts**: From Brendan Gregg's repository

Process:

1. Profile application: `perf record -g -F 999 ./your_program`
2. Generate FlameGraph: Convert perf.data to SVG visualization
3. Analyze results:
    - Wide stacks = high CPU usage
    - Deep stacks = long call chains
4. Optimize identified bottlenecks and retest

Note: Time-intensive; prioritize if performance issues are known. Document findings in PERFORMANCE.md.

CPU Scheduling Simulator

Task 4: Git Visualization with Gource

Objective: Create dynamic visual representation of Git history for presentations and potential bonus points.

Benefits:

- Shows team contributions over time
- Engaging way to showcase project evolution
- Interactive timelines of file changes and branch activity

Steps:

1. Install Gource
2. Generate visualization with customized settings (title, speed, user highlights)
3. Record output as MP4 video using ffmpeg
4. Share with team and add regeneration instructions to docs

https://github.com/Masrkai/OS_Project.git